Triage — 5 Minute Finalist Demo Script

ONE LINE TO REMEMBER IF YOU GET LOST

> "Triage figures out who is calling your API, how much to trust them, and what they should pay."

THE SCRIPT

0:00–0:25 — Problem

"AI agents are going to hit every API on the internet. The problem is that today, APIs can't tell if a request comes from a verified human, a human-backed agent, or an anonymous bot — and they can't price them differently. Triage is the trust layer that solves that."

[open dashboard tab — have it ready before you start]

0:25–1:35 — Live Dashboard Demo

"Watch this live."

[fire curl commands one by one, pause after each so they see the dashboard react]

```bash
# 1. Verified human — GREEN
curl -H "x-world-id: verified" https://triage-hackathon-production.up.railway.app/ap

# 2. Human-backed agent — BLUE
curl -H "x-agentkit-demo: human-backed" -H "x-agent-address: 0xAGENT" https://triage

# 3. Anonymous bot — YELLOW
curl -H "x-agent-address: 0xBOT1234" https://triage-hackathon-production.up.railway.

# 4. Nothing — RED, BLOCKED
curl https://triage-hackathon-production.up.railway.app/api/data
```

"Each request gets classified live, scored for trust, and priced accordingly. Green is free. Blue pays less. Yellow pays full price. Red is blocked."

1:35–2:15 — Trust Formula

"The trust score has four parts: identity, behavior, reputation, and risk.

- **Identity** is the base — are you a World ID human, an AgentKit agent, or anonymous?
- **Behavior** measures your success rate on this API over time
- **Reputation** rewards consistency — how long have you been around?
- **Risk** subtracts trust for suspicious patterns like payment failures or request spikes

Higher trust means lower fees. It's inspired by EigenTrust from Stanford — adapted for agent traffic and pricing."

[point to the trust leaderboard on the dashboard]

"You can see agents building trust in real time here."

2:15–3:05 — The Roaster + npm Package

[switch to Roaster tab]

"This is a completely separate website — The Roaster. It installed triage-middleware from npm. Same trust layer, same classification, same dashboard."

[click Get Roasted without verifying] "No verification — blocked."

[verify with World ID] "Now I verify with World ID through the World App."

[click Get Roasted again] "Free access. You can see it hit the dashboard as a verified human."

[navigate to /triage on the Roaster] "And the dashboard mounted automatically at /triage — came with the package."

3:05–3:35 — Code Snippet

"This is all it takes to add Triage to any Hono API:"

```typescript
import { triage, triageDashboard, attachWebSocketToServer } from 'triage-middleware'

app.use('/api/*', triage({ payTo: '0xYourWallet' }))

triageDashboard(app, { rpId: 'rp_...', signingKey: '0x...' })

const server = serve({ fetch: app.fetch, port: 3000 })
attachWebSocketToServer(server)
```

"Three imports. Four lines. You get classification, trust scoring, dynamic x402 pricing, and a live monitoring dashboard at /triage. Automatically."

3:35–4:05 — Marketplace

[switch to marketplace tab]

"Once trust is a primitive, you can build on top of it. This marketplace is one example — agents can be hired for tasks. Trust score determines their pricing tier and ranking. Higher trust, higher earning potential, lower participation fee."

[show agent cards briefly, don't deep dive]

"The same trust layer that protects APIs also powers agent discovery and hiring."

4:05–4:20 — Communities (mention only)

"The same infrastructure also supports content monetization — humans get free access, agents pay. But the core today is the middleware and the marketplace."

4:20–5:00 — Close

"So Triage is trust infrastructure for the agent economy. It classifies every request, scores trust, prices access dynamically, and gives you real-time visibility into everything — all in one npm package.

World ID for humans. AgentKit for human-backed agents. x402 for payments. EigenTrust for scoring.

Every API will need this as agent traffic explodes. We built it this weekend."

READY-MADE Q&A

"How is this different from AgentScore?"

"AgentScore does trust scoring but has no World ID integration. AgentKit does proof-of-human but doesn't track behavior or pricing. Triage bridges both — identity plus reputation plus dynamic pricing in one middleware. We're the layer that connects the two ecosystems."

"How does the npm package actually work?"

"It wraps your API routes before your handlers run. On each request it classifies the caller, computes a trust score, maps that to a price, sets response headers, and emits a real-time event to the dashboard. The dashboard is bundled inside the package — triageDashboard() serves it at /triage automatically. You don't build anything."

"What's actually real right now vs demo?"

"Real today: World ID verification with server-verified nullifier hashes, x402 payment specs with real USDC flow on Base Sepolia, trust scoring, dashboard events, and the published npm package. Partial today: full AgentKit on-chain contract verification and persistent storage — those are the first v2 upgrades."

"Why is the max trust score 90 and not 100?"

"Perfect trust should be unattainable by design. Even the most consistent agent carries some uncertainty. 90 is the ceiling — you can get close, but never perfect."

"What happens if the x402 facilitator goes down?"

"We fail closed — unverified requests get blocked, not passed through. That's the correct behavior for a trust system. You never want to fail open."

"Is the data persistent?"

"In-memory for the hackathon — JavaScript Maps and Sets. PostgreSQL is the first item on the v2 roadmap. Scores survive the demo but reset on server restart."

"Why Base Sepolia and not mainnet?"

"Hackathon constraint. V3 roadmap has Base mainnet with an audited escrow contract for marketplace payments."

"What about the XMTP bot?"

"Works locally. There's a known Railway deployment issue with tsx and native Node bindings for @xmtp/node-bindings. Server continues without it — XMTP is non-blocking. First fix in v2."

"Can this work with Express or Next.js, not just Hono?"

"Not yet — Hono only for v1. Express, Fastify, Next.js middleware, and Cloudflare Workers adapters are on the v2 roadmap."

"What if someone spoofs the x-world-id header on the npm package?"

"The npm package doesn't accept demo headers — only server-verified nullifier hashes from real World ID proofs. The demo headers only exist on the main demo server for reliable hackathon flows. The published package is strict."

"How does dynamic pricing actually work with x402?"

"When a non-human hits the endpoint without payment, we return a 402 with the exact USDC amount calculated from their trust score. They pay that amount, retry with the payment signature, and get through. The price is embedded in the 402 response spec."

"Why would a developer use this instead of just blocking all bots?"

"Because blocking all bots also blocks legitimate agent traffic — a user's AI assistant trying to use

your API. Triage gives you a third option: classify traffic by trust, charge unknown agents, and keep it free for verified humans. You monetize the agent economy instead of fighting it."

TRANSITIONS TO PRACTICE

These are where live demos usually break. Practice each one:

1. **Problem → Dashboard**: have dashboard open before you start talking
2. **Curl → Dashboard reaction**: pause after each curl, point at the screen
3. **Dashboard → Roaster**: have Roaster tab already open
4. **Roaster → Code snippet**: have VS Code or a code slide ready
5. **Code snippet → Marketplace**: have marketplace tab open

TAB ORDER (have these open before you start)

1. Dashboard — triage-hackathon-production.up.railway.app
2. The Roaster — triage-roaster-production.up.railway.app
3. The Roaster /triage — triage-roaster-production.up.railway.app/triage
4. Terminal (for curl commands)
5. Code snippet (VS Code or a notes file)